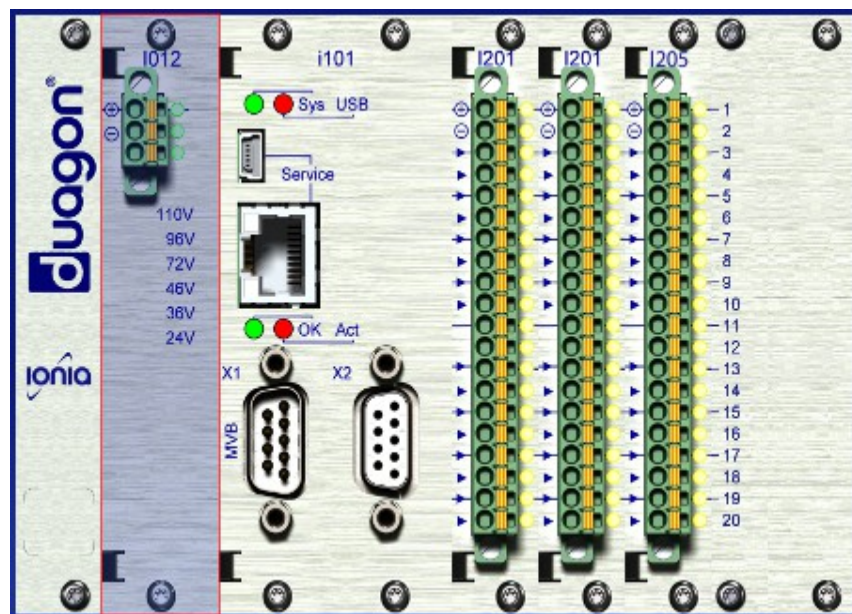




Ionia Get started Documentation

Set-up and Configuration

ionia - a Programmable Logic Controller (PLC) with configurable number of digital/analog input and output channels and fieldbus communication modules. The CPU module can be selected with a fieldbus interface of either MVB, CAN or Ethernet.



duagon AG
Riedstrasse 12
CH-8953 Dietikon
Phone +41 44 743 73 00
Fax +41 44 743 73 15

www.duagon.com

duagon[®]
Moving TCMS Solutions

Duagon Data Sheet Preamble

On having purchased products described in this data sheet, the customer acquires the right to use the products according to its specified purpose and in accordance with all operation, service and maintenance instructions. All other rights to the product, Duagon's intangible assets rights in particular, belong solely to Duagon and may not be deemed to have been assigned along with the sale of the products.

All product properties are fully described in the data sheet under express exclusion of any warranty for other properties. Of decisive relevance is the data sheet valid at the time of the order being placed. Duagon provides a warranty that the product properties are retained during the period of warranty. Evidence that the properties of the product have been retained will be brought, always and exclusively, on Duagon premises by means of a test construction pursuant to the type test.

The customer is obliged to inspect whether the products themselves are suitable for the application intended. In particular, that inspection must include the integration of the products into the intended system configuration and a check on whether the properties as per data sheet can be fulfilled once integrated into the system configuration as planned by the customer. Since the products are not certificated for operation with security applications, the customer must take appropriate measures to ensure that any malfunctions that may occur in a system configuration with other products will be absorbed by supplementary security measures.

The period of warranty for the products is 24 months and it begins on the date the products are shipped from the factory.

The warranty that Duagon assumes for the products will, at Duagon's discretion, be limited either to the repair of or the replacement of the products at the Duagon factory. The warranty solely covers the products or parts thereof which, despite professional handling, have become defective or unusable and which arrive at the Duagon factories for repair or replacement during the period of warranty. The extent of Duagon's warranty is fully set out in this data sheet. Duagon cannot be held liable for consequential damage caused by a defect or for indirect damage or for consequential damage of any kind. Therefore the customer bears all and any costs that occur due to production downtime, for example, or due to the installation or dismantling of products or due to their transportation to Duagon and back.

Duagon's liability and warranty do not obtain if evidence cannot be brought that the products were being operated according to its specified purpose and in accordance with all operation, service and maintenance instructions as issued by Duagon.

These provisions form an integrated part of the product properties. Duagon products cannot be acquired with other or more extensive degrees of warranty and liability on the part of Duagon.

This data sheet is to be evaluated in accordance with **Swiss law**. The court of jurisdiction is the **seat of the vendor**. The applicability of the UN agreement as to international sales of goods (also known as "Viennese Purchasing Convention") is herewith expressly excluded.

duagon AG, Riedstrasse 12, CH-8953 Dietikon, Switzerland

Phone: +41 44 743 73 00, Fax: +41 44 743 73 15, www.duagon.com

Document history:

Rev.	Creator Date	Creator Name	Released Date	Released Name	Comments	Ident-Number
Draft	13-06-03	Bollinger	06.06.2013	Ch.Bollinger	See chapter Document history	d-007724-014426
1	13-08-27	Dent	29.08.2013	J. Dent		d-007724-015008
2	13-09-11	Bollinger	10.02.2014	Ch.Bollinger		d-007724-015105

Table of Contents

1. General Safety Summary.....	4
2. Introduction.....	5
2.1 System Architecture Overview.....	5
3. Hardware Setup.....	6
3.1 Layout.....	6
3.2 Module numbering convention.....	6
3.3 Power Connection.....	7
4. Software Setup.....	8
4.1 MULTIPROG Express Installation.....	8
4.2 DHCP Server.....	9
5. Pre-Installed Sample Applications.....	10
5.1 Selecting a sample application.....	10
5.2 Connection to the Target System.....	10
5.3 Compiling the Application.....	11
5.4 Controlling the Application.....	11
5.5 MVB Application.....	11
5.5.1 i101_MVB_App_001 Multiprog Application.....	11
5.5.2 Duagon MVB Driver.....	12
5.5.3 MVB Port Configuration.....	13
5.6 ETH Application.....	16
5.6.1 i102_UDP_App_001 Multiprog Application.....	16
5.6.2 Duagon Firmware Libraries (for UDP).....	16
5.6.3 Ethernet Application.....	16
5.6.4 ETH Port Configuration.....	17
5.6.5 Using the Ethernet Application.....	17
5.7 WTB Gateway Application: D521_WTB_GW_App_001.....	19
5.7.1 Communication Concept.....	19
5.7.2 MVB Configuration.....	20
5.7.3 Programming Logic.....	21
5.7.4 WTB Configuration.....	23
6. IP configuration.....	25
6.0.1 IP configuration from within an IEC61131 program.....	25
7. References.....	27
8. Document history.....	28
Appendix A: Document Numbering System.....	29

1. General Safety Summary

Review the following safety precautions to avoid injury and prevent damage to this product or those connected to it. To avoid potential hazards, use this product only as specified.

Only qualified personnel should perform installation, maintenance and service procedures

To avoid fire or personal injury

Connect and disconnect properly. This data sheet contains all relevant information for connecting the device.

Power On only with all connections made. All connectors on the device must be connected (unused connections should be covered with a dummy connector).

Ground the product. Ground connection is located on the device.

Observe all terminal ratings. To avoid fire or shock hazard, observe all ratings and markings on the product. Consult this data sheet before making connections to the product.

Avoid exposed circuitry. Do not touch exposed connections and components when power is present.

Do not operate with suspected failures. If you suspect there is a damage to this product, have it inspected by qualified service personnel, or return it to Duagon AG.

Safety Terms and Symbols

Terms in this data sheet. These terms may appear in this data sheet.



WARNING. Warning statements identify conditions or practices that could result in injury or loss of life.



CAUTION. Caution statements identify conditions or practices that could result in damage to this product or other property.

Service Safety Summary

Only qualified personnel should perform installation, maintenance and service procedures

Do not service with Power On. Dangerous voltages or currents may exist in this product. Disconnect power, remove battery (if applicable), and disconnect test leads before removing protective panels, soldering, or replacing components.

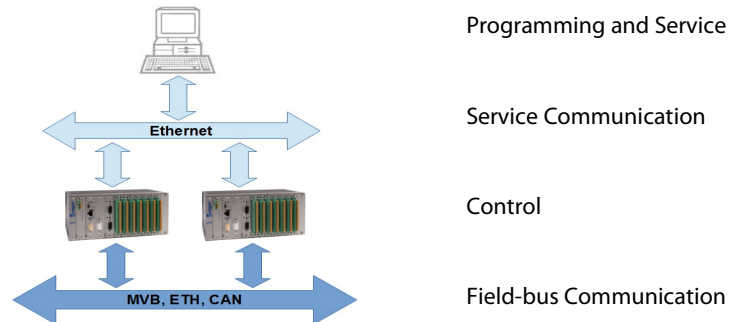
To avoid electric shock, do not touch exposed connections.

2. Introduction

This document describes the necessary steps, used to set up an ionia TCMS system. After following the described steps you will have a full working ionia TCMS device with its IEC61131 programming environment installed.

2.1 System Architecture Overview

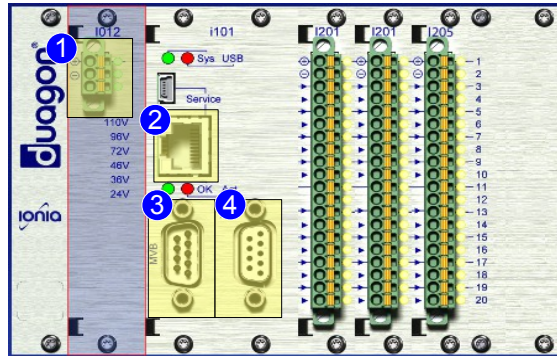
In an ionia TCMS system architecture, the following items can be distinguished:



- **Programming and Service**
 - A windows based personal computer used as workstation with Multiprog Express programming environment installed.
 - With the Multiprog Express programming environment the TCMS application is programmed in one of the 5 programming languages defined in the IEC61131-3 standard.
 - The configuration of the Field-bus is performed by means of a Field-bus specific configuration tool.
 - In order to access the ionia TCMS system over Ethernet by TCP/IP, a DHCP server installed on this workstation shall provide the desired IP address to the ionia TCMS system.
- **Service Communication**
 - The service communication is used for download and online debugging of the TCMS application.
- **Control**
 - A powerful ARM Cortex A8 based CPU executes the TCMS application program.
 - The binary and analogue input channels are read into the TCMS application and processed before being written to the output channels.
 - The same procedure as described for periphery signals is applied to the field-bus signals
- **Field-bus communication**
 - The field-bus communication is applied according to the actual configuration. The interface to the TCMS application consists of the transmitted and received signals, only.

3. Hardware Setup

3.1 Layout



- ① Power Connector
- ② Ethernet Service Connector
- ③ MVB Fieldbus Connector X1 (male)
- ④ MVB Fieldbus Connector X2 (female)

Note: this example shows the i01, the i02 ETH and i03 CAN have different connectors at ③ and ④ .

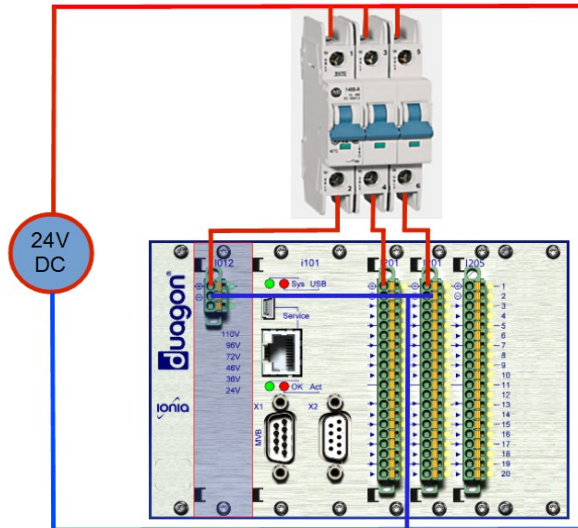
3.2 Module numbering convention

The following module numbering scheme is respected for the ionia modules:

- i012-LV: Module Nr 00
- i01: Module Nr 01
- i201: Module Nr 02
- i202: Module Nr 03
- i205: Module Nr 04

3.3 Power Connection

For powering the ionia device, please follow the drawing below.



Note:

1. It is recommended to use independent power circuits to supply the CPU and each i201 rep. i202 card.
2. Binary output card i205 must not be powered by an external power source.

4. Software Setup

4.1 MULTIPROG Express Installation

In order to guide you with the software installation an installer is provided.

MULTIPROG_Express_d-007271-014411

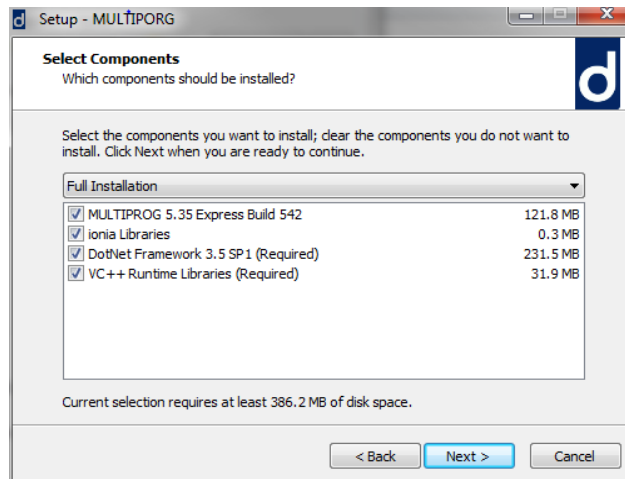
The Multiprog Express installer provides you with:

- Multiprog Express 5.35
- ionia Libraries
- DotNet Framework 3.5 SP1
- VC++ Runtime Libraries

The first dialog of the installer shows the version information of the installer itself, which serves as identification of the running software.

The next dialog asks for a directory where to install Multiprog Express, if you have no preferences then the default setting is OK.

The third dialog shows all available installation options:



These options can be activated/deactivated by means of setting a flag in the corresponding check box. There are also predefined installation template which you can choose from, by selection a corresponding option in the selection field.

Remark:

If on your computer a component is already installed, than there is no need to install it again, e.g. if on your system DotNet Framework 3.5. SP1 is already installed then you must not select this option.

If you decide at runtime of the installer, that e.g. DotNet Framework 3.5. SP1 should not be installed then you can press Cancel the corresponding dialog.

4.2 DHCP Server

A first installation of an ionia TCMS device requires a DHCP server, from which ionia system gets its ip address. This ip address must be known by you. There are various possibilities to master this:

1. If you have access to the ip table of your DHCP server, i.e. you are the network administrator, you can use the DHCP server of your local network.
2. Install a DHCP server on your windows machine which provides the ip address. Various free DHCP servers can be found on the internet.
3. Use a router with a DHCP server service and find the provided ip address by means of your routers Webserver.
4. Install Linux in a virtual machine and activate the DHCP server, e.g. ISC DHCP within Linux. With the following command:
 - `dhcpd eth2 -d -f`
 - eth2 is the interface to your sub-domain.

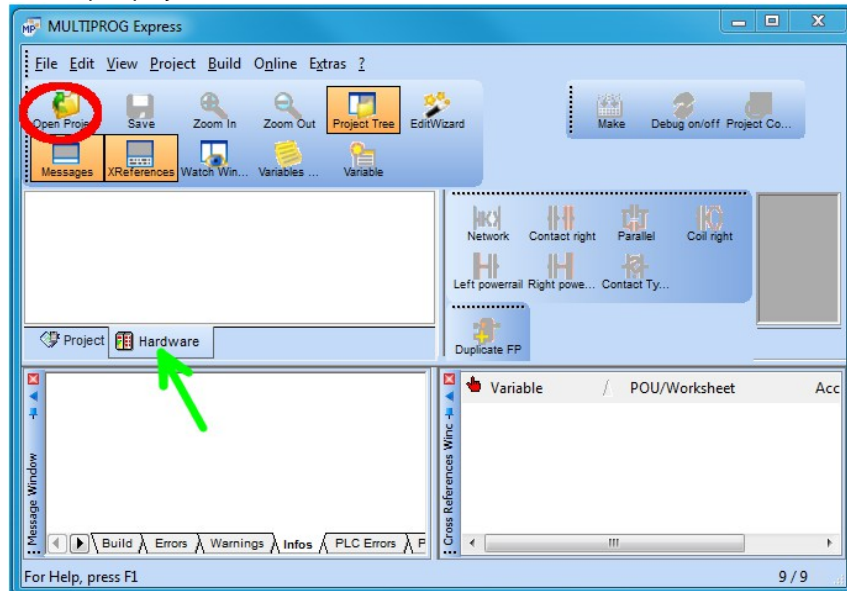
are the log messages sent to the console, thus you can observe which ip address is provided.

Further instructions are given in the man page dhcp.

5. Pre-Installed Sample Applications

5.1 Selecting a sample application

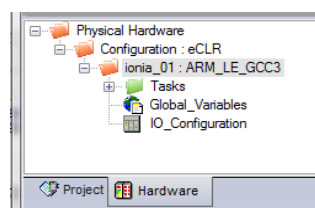
Select open project:



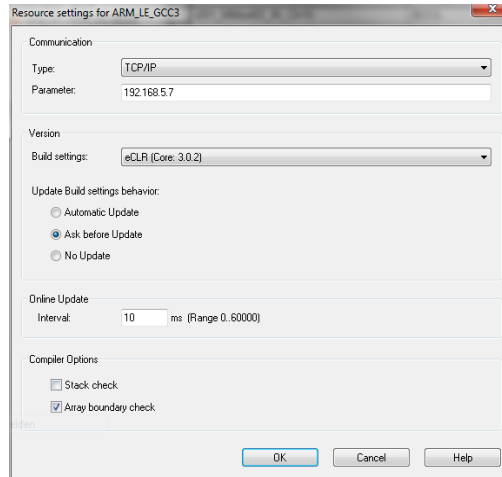
In the Open/Unzip project dialog select an application for example i102_UDP_App_001.mwe or i101_MVB_App_001.mwe as appropriate. When the project has loaded select the hardware tab.

5.2 Connection to the Target System

The configuration of the programming environment and target system is done within the hardware editor by means of a right click on the resource icon and then selecting Settings.

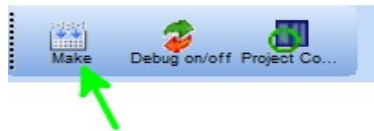


Within the Resource settings dialog the ip address of the ionia system has to be set to the correct value. e.g.:



5.3 Compiling the Application

If any changes are made to an application it will have to be compiled before it can

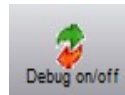


be downloaded. To do this use the Make button.

5.4 Controlling the Application



Start, Stop and Download an application Program. Click on this button to download a Multiprog application to the ionia system. After a download you will be able to start and stop your application from the corresponding window.



Online debugging with a connected device. Click on this button to start debugging your Multiprog application: signal values are displayed and can be forced to some desired value.

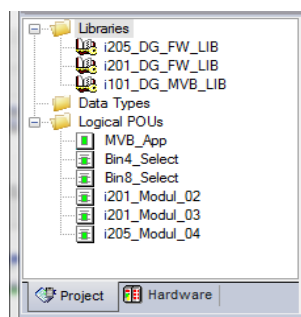
5.5 MVB Application

5.5.1 i101_MVB_App_001 Multiprog Application

The "i101_MVB_App_001" application maps all periphery in- and output signals of the ionia TCMS System to corresponding MVB ports. For detailed documentation regarding the MVB mapping, please refer to chapter 5.5.3.

Duagon Firmware Libraries

The duagon firmware libraries used to program the periphery of the ionia TCMS System are supplied together with the installer.



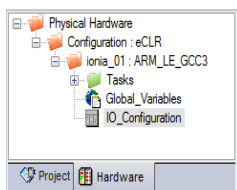
Within the **Project Editor** of the Multiprog Express programming environment the libraries used with the i101_MVB_App_001 are displayed.

By means of a right mouse-click on the libraries symbol additional libraries can be added.

The default location where the delivered libraries are stored is as follows:

- c:\ProgramData\KW-Software\MULTIPROG Express\5_35_542\plc\FW_LIB\

5.5.2 Duagon MVB Driver



By means of a double click on the IO_Configuration icon it is possible to load and configure the duagon MVB Driver.

For the addressing of the signals with respect to the port configuration, refer to chapter "5.5.3 MVB Port Configuration". This addressing is done in the Global_Variable worksheet.

	Name	Type	Usage	D...	Address
	I201 Module_02 Input Group				
	MVB_i201_Mod02_isUser	BOOL	VAR_GLOBAL		%IX256000.0
	MVB_i201_Mod02_LED_Ch01	BOOL	VAR_GLOBAL		%IX256000.1
	MVB_i201_Mod02_LED_Ch02	BOOL	VAR_GLOBAL		%IX256000.2
	MVB_i201_Mod02_LED_Ch03	BOOL	VAR_GLOBAL		%IX256000.3
	MVB_i201_Mod02_LED_Ch04	BOOL	VAR_GLOBAL		%IX256000.4
	MVB_i201_Mod02_LED_Ch05	BOOL	VAR_GLOBAL		%IX256000.5

The definition of the address itself follows the IEC-61131 standard with the following syntax:

- Location Prefix / Size Prefix / IO Address [. Byte Offset]

Location Prefix

- I : Input
- Q : Output

Size Prefix

- X : 1 Bit Value
- B : 8 Bit Value
- W : 16 Bit Value

IO Address

- 5 numerical digits for Port (decimal number)
- 3 numerical digits for Byte offset (decimal number)
- Example: Address 1280001 → Port 1280 / Byte Offset 001

5.5.3 MVB Port Configuration

The MVB port configuration described in this section is the configuration used by the “i101_MVB_App_001” application.

MVB Sink Ports

Port 0x100 – i202 Module Nr. 02 outputs

- Periphery output signals.
- Interface to the user LEDs of Module Nr. 02.

1								0								Byte Offset
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	Bit Offset
MVB_i201_Mod02_LED_Ch16	MVB_i201_Mod02_LED_Ch15	MVB_i201_Mod02_LED_Ch14	MVB_i201_Mod02_LED_Ch13	MVB_i201_Mod02_LED_Ch12	MVB_i201_Mod02_LED_Ch10	MVB_i201_Mod02_LED_Ch09	MVB_i201_Mod02_LED_Ch08	MVB_i201_Mod02_LED_Ch07	MVB_i201_Mod02_LED_Ch06	MVB_i201_Mod02_LED_Ch05	MVB_i201_Mod02_LED_Ch04	MVB_i201_Mod02_LED_Ch03	MVB_i201_Mod02_LED_Ch02	MVB_i201_Mod02_LED_Ch01	MVB_i201_Mod02_isUser	Signal Name

Port 0x101 – i202 Module Nr. 03 outputs

- Periphery output signals.
- Interface to the user LEDs of Module Nr. 03.

1								0								Byte Offset
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	Bit Offset
MVB_i201_Mod03_LED_Ch16	MVB_i201_Mod03_LED_Ch15	MVB_i201_Mod03_LED_Ch14	MVB_i201_Mod03_LED_Ch13	MVB_i201_Mod03_LED_Ch12	MVB_i201_Mod03_LED_Ch10	MVB_i201_Mod03_LED_Ch09	MVB_i201_Mod03_LED_Ch08	MVB_i201_Mod03_LED_Ch07	MVB_i201_Mod03_LED_Ch06	MVB_i201_Mod03_LED_Ch05	MVB_i201_Mod03_LED_Ch04	MVB_i201_Mod03_LED_Ch03	MVB_i201_Mod03_LED_Ch02	MVB_i201_Mod03_LED_Ch01	MVB_i201_Mod03_isUser	Signal Name

Port 0x102 – i205 Module Nr. 04 outputs

- Periphery output signals.
- Interface to the user LEDs of Module Nr. 04.
- Interface to output ports of Module Nr. 04.

1								0								Byte Offset
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	Bit Offset
MVB_I205_Mod04_OUT_Ch03	MVB_I205_Mod04_OUT_Ch02	MVB_I205_Mod04_OUT_Ch01	MVB_I205_Mod04_LED_Ch12	MVB_I205_Mod04_LED_Ch11	MVB_I205_Mod04_LED_Ch10	MVB_I201_Mod04_LED_Ch09	MVB_I201_Mod04_LED_Ch08	MVB_I201_Mod04_LED_Ch07	MVB_I201_Mod04_LED_Ch06	MVB_I201_Mod04_LED_Ch05	MVB_I201_Mod04_LED_Ch04	MVB_I201_Mod04_LED_Ch03	MVB_I201_Mod04_LED_Ch02	MVB_I201_Mod04_LED_Ch01	MVB_I201_Mod02_isUser	Signal Name

2																Byte Offset
7	6	5	4	3	2	1	0									Bit Offset
Not used	Not used	Not used	MVB_I205_Mod04_OUT_Ch08	MVB_I205_Mod04_OUT_Ch07	MVB_I205_Mod04_OUT_Ch06	MVB_I205_Mod04_OUT_Ch05	MVB_I205_Mod04_OUT_Ch04									Signal Name

MVB Source Ports

Port 0x200 – i202 Module Nr. 02 inputs

- Periphery input signals of Module Nr 02

1								0								Byte Offset
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	Bit Offset
i201_MMMod02_IN_Ch16	i201_MMMod02_IN_Ch15	i201_MMMod02_IN_Ch14	i201_MMMod02_IN_Ch13	i201_MMMod02_IN_Ch12	i201_MMMod02_IN_Ch11	i201_MMMod02_IN_Ch10	i201_MMMod02_IN_Ch09	i201_MMMod02_IN_Ch08	i201_MMMod02_IN_Ch07	i201_MMMod02_IN_Ch06	i201_MMMod02_IN_Ch05	i201_MMMod02_IN_Ch04	i201_MMMod02_IN_Ch03	i201_MMMod02_IN_Ch02	i201_MMMod02_IN_Ch01	Signal Name

3								2								Byte Offset
15								0								Bit Offset
i201_Mod02_IN_ERROR																Signal Name

5								4								Byte Offset
15								0								Bit Offset
i201_Mod02_LCONFIG_ERROR																Signal Name

7								6								Byte Offset
15								0								Bit Offset
i201_Mod02_LSET_ERROR																Signal Name

Port 0x201 – i202 Module Nr. 03 inputs

- Periphery input signals of Module Nr 03

1								0								Byte Offset
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	Bit Offset
i201_MMMod03_IN_Ch16	i201_MMMod03_IN_Ch15	i201_MMMod03_IN_Ch14	i201_MMMod03_IN_Ch13	i201_MMMod03_IN_Ch12	i201_MMMod03_IN_Ch11	i201_MMMod03_IN_Ch10	i201_MMMod03_IN_Ch09	i201_MMMod03_IN_Ch08	i201_MMMod03_IN_Ch07	i201_MMMod03_IN_Ch06	i201_MMMod03_IN_Ch05	i201_MMMod03_IN_Ch04	i201_MMMod03_IN_Ch03	i201_MMMod03_IN_Ch02	i201_MMMod03_IN_Ch01	Signal Name

3	2	Byte Offset
15	0	Bit Offset
i201_Mod03_IN_ERROR		Signal Name

5		4		Byte Offset
15		0		Bit Offset
i201_Mod03_LCONFIG_ERROR				Signal Name

7	6	Byte Offset
15	0	Bit Offset
i201_Mod03_LSET_ERROR		

Port 0x202 – i205 Module Nr 04 inputs

- Periphery input signals of Module Nr. 04

1								0								Byte Offset
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	Bit Offset
Not used	Not used	Not used	Not used	Not used	Not used	Not used	Not used	i205_Mod04_Relay_ERROR	i205_MMMod04_IO_Ch08	i205_MMMod04_IO_Ch07	i205_MMMod04_IO_Ch06	i205_MMMod04_IO_Ch05	i205_MMMod04_IO_Ch04	i205_MMMod04_IO_Ch03	i205_MMMod04_IO_Ch02	Signal Name

3																2																Byte Offset																																															
15																																																0																Bit Offset															
i205_Mod04_IO_ERROR																																																																Signal Name															

5		4		Byte Offset
15				0 Bit Offset
i205_Mod04_LSET_ERROR				Signal Name

7				6				Byte Offset							
15								0				Bit Offset			
i205_Mod04_LCONFIG_ERROR															

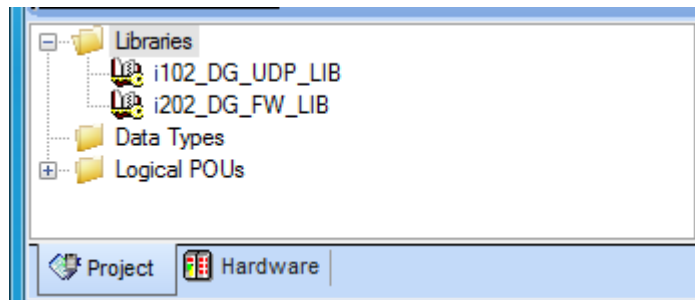
5.6 ETH Application

5.6.1 i102_UDP_App_001 Multiprog Application

The i102 UDP application implements a simple Ethernet control application. The third data byte of a UDP datagram is fed via a simple decoder to an i202 digital output module. For more details see 5.6.3

5.6.2 Duagon Firmware Libraries (for UDP)

The duagon firmware libraries used to program the periphery of the ionia TCMS System are supplied together with the installer.



Within the **Project Editor** of the Multiprog Express programming environment the libraries used with the i102_UDP_App_001 are displayed.

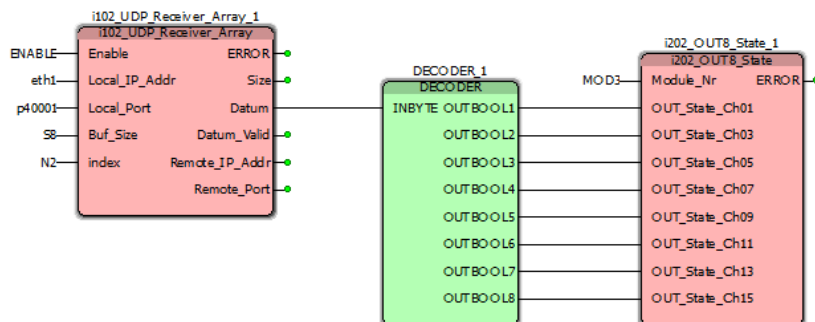
By means of a right mouse-click on the libraries symbol additional libraries can be added.

The default location of the delivered libraries is as follows:

- c:\ProgramData\KW-Software\MULTIPROG Express\5_35_542\plc\FW_LIB\

5.6.3 Ethernet Application

The program is designed as a simple illustrative working example to promote understanding of the function blocks used.



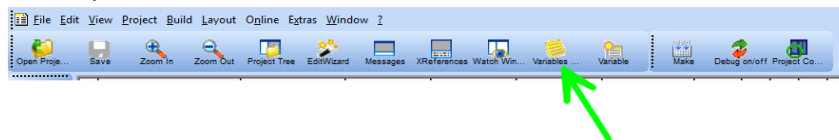
The i102_UDP_Receiver_Array receives UDP datagrams addressed to the local port on the interface (usually eth1) with a specified IP address. The data bytes of the UDP datagram are stored in an internal array. The individual elements of the array can be accessed by use of the index input and the datum output. The decoder sets one of its outputs TRUE for each 1 bit in the INBYTE. (Double click on the decoder in MULTIPROG to see its internal logic).

5.6.4 ETH Port Configuration

The variable eth1 holds the Ethernet address of the interface on which a listening port will be opened when the FB is enabled. The port number is determined by variable p40001. The variable MOD3 assumes that the third module in the ionia is i202 digital IO module.

Name	Type	Usage	Description	Address	Init
Default					
i102_UDP_Receiver_Array_1	i102_UDP_Recei...	VAR			
i202_OUT8_State_1	i202_OUT8_State	VAR			
DECODER_1	DECODER	VAR			
ENABLE	BOOL	VAR			TRUE
eth1	STRING	VAR			'192.168.1.40'
p40001	UINT	VAR			40001
S8	DINT	VAR			8
N2	UINT	VAR			2
MOD3	INT	VAR			3

To change the Ethernet address, port number or module number the variables table can be opened via the toolbar as follows:



Note to change the Ethernet address used by the ionia TCMS see: IP configuration

5.6.5 Using the Ethernet Application

To try out the application the following python script¹ can be run on a PC connected to same network as the ionia:

```
#!/ python2
import socket as s
import time as t

try:
    tr.close()
except:
    pass

port = 40001
buffer_size = 80000
tr=s.socket(s.AF_INET, s.SOCK_DGRAM)
tr.bind(("192.168.1.191", 22003))
inadd = "192.168.1.40"

while(1) :
    data = "--%s" % (chr(0xAA))
    tr.sendto(data, (inadd, port))
    t.sleep(0.3)
    print "sent %s" % data
    data = "--%s" % (chr(0x55))
    tr.sendto(data, (inadd, port))
    t.sleep(0.5)
    print "sent %s" % data
```

¹If necessary python can be installed from <http://www.python.org/download/releases/2.7.5/>.

The line `tr.bind(("192.168.1.191", 22003))` contains the Ethernet address of the interface on the PC connected to the same network as eth1 on the ionia. It will probably have to be changed to the correct address or the PC interface address will have to be set accordingly.

When the script is started alternate outputs of the i202 module should toggle on and off as indicated by the LEDs.

5.7 WTB Gateway Application: D521_WTB_GW_App_001

The WTB Gateway Application enables the ionia system to actively take part on the communication at train bus level, according to TCN/IEC 61375. Since the TCN standard does not define an application profile, it is up to the application programmer to implement an application profile according its needs. Predefined application profile such as the UIC-556 can be implemented.

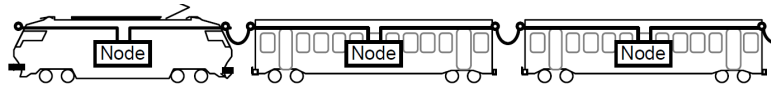


Figure 1 – Wire Train Bus

Figure 1 illustrates the general idea of the WTB application architecture: A WTB node communicate to its consist network and to its partner nodes on the WTB.

5.7.1 Communication Concept

The layout of the Gateway Application Sample D521_WTB_GW_App_001 is ready to communicate with two WTB nodes and a partner device on the MVB consist network.

Of course, more devices could be attached to the consist network as well as to the WTB network. The limiting factor of using two nodes only is the process data mapping implemented in the Gateway Application Sample itself. The process data mapping can be reprogrammed easily by an IEC61131 application programmer.

After the WTB Inauguration, the process data communication over WTB is established, whereas the receiving resp. transmitting data is send resp. read from the corresponding MVB port.

The notation of SRC and Sink port is always with respect to the attached media, i.e MVB resp. WTB.

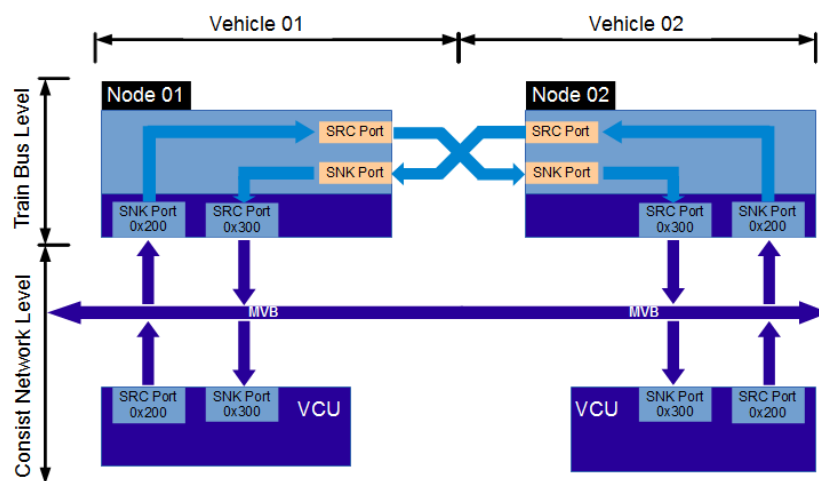


Figure 2: Communication Concept

The WTB Gateway Architecture (Figure 3) shows the programming building blocks of a WTB Nodes. All these parts can be reprogrammed resp. reconfigured according to application specific needs.

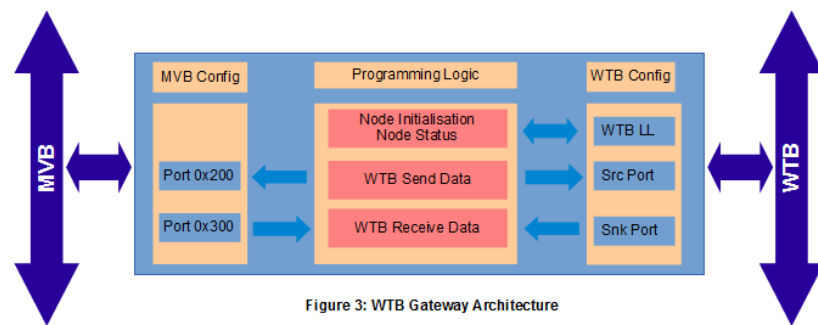


Figure 3: WTB Gateway Architecture

In the following are the default settings of the configurable and programmable building blocks described (see figure 3).

5.7.2 MVB Configuration

The design of the MVB configuration is done by means of the duagon D2000 tool. A MVB description file serves as input of the command line tool D2000. This text file is parsed by D2000 and generates among others an appropriate bin file as output. i.e. ionia_01.bin.

The ionia_01.bin file is located in the MULTIPROG project directory:

%MP_PROJE%\projects\D521_WTB-GW_App_001\C\Configuration\R\ionia_01\

The according process variables can be found in MULTIPROG in the Global_Variables worksheet.

Port 0x200

Name	pd_tx
Size	32 Byte
Type	Sink Port

Port 0x200 receives the process data from the consist network and forwards them to the WTB.

Since on the receiver side (Port 0x300) the GW application writes the node strength into byte 0, data byte 0 should not be used otherwise the data will be lost.

The Process variables of Port 0x200 can be found in the Global_Variables worksheet of MULTIPROG (see below).

Name	Type	Usage	Description	Address
⊕ Type_NodeDescriptor				
⊕ Type_Node_Status				
⊖ End Type_WTB_Status				
⊕ Type_Topography				
⊖ End Type_Topography				
⊕ Various TCN Types				
⊕ Service				
⊕ Link Layer Configuration				
⊕ PD Communication				
⊕ MVB COMMUNICATION				
⊖ Port 0x200 PD WTB Send (Sink)				
MVB_WTB_Send_data_byte_000	USINT	VAR_GLOBAL		%IB0512000
MVB_WTB_Send_data_byte_001	USINT	VAR_GLOBAL		%IB0512001
MVB_WTB_Send_data_byte_002	USINT	VAR_GLOBAL		%IB0512002
MVB_WTB_Send_data_byte_003	USINT	VAR_GLOBAL		%IB0512003

The addressing is as described in 5.5.2 Duagon MVB Driver.

Port 0x300

Name	pd_rx
Size	32 Byte
Type	Source Port

The process data received from the other node over the WTB are sent to the consist network by means of Port 0x300.

The first data byte of Port 0x300 contains the strength of the node:

- 0x01 Strong Master
- 0x02 Weak Master
- 0x03 Regular Slave

The Process variables of Port 0x300 can be found in the Global_Variables worksheet of MULTIPROG (see below).

Name	Type	Usage	Description	Address
System Variables				
Link Layer Service L_RESULT				
Report				
Type_WTBStatus				
Link Layer Service State				
End Link Layer Service State				
Type_NodeDescriptor				
Type_Node_Status				
End Type_WTB_Status				
Type_Topography				
End Type_Topography				
Various TCN Types				
Service				
Link Layer Configuration				
PD Communication				
MVB COMMUNICATION				
Port 0x200 PD WTB Send (Sink)				
Port 0x300 PD WTB Receive (Source)				
WTB_Receive_data_byte_000	USINT	VAR_GLOBAL		%QB768000
WTB_Receive_data_byte_001	USINT	VAR_GLOBAL		%QB768001
WTB_Receive_data_byte_002	USINT	VAR_GLOBAL		%QB768002
WTB_Receive_data_byte_003	USINT	VAR_GLOBAL		%QB768003

The addressing is as described in 5.5.2 Duagon MVB Driver.

5.7.3 Programming Logic

The programming logic serves four purposes:

- Node Initialisation.
- Read out node status.
- Prepare the process data to be sent to the WTB.
- Post processing of the received process data from the WTB.

All these tasks are programmed within the IEC 61131 programming environment MULTIPROG Express and are explained in more detail below.

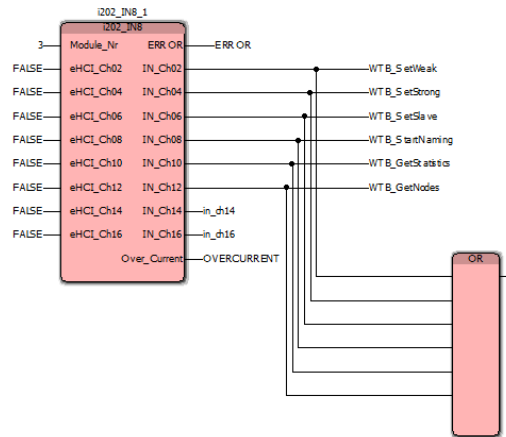
Node Initialisation and Status

The node initialisation is contained in the **Programming Organisation Unit (POU) WTB_Main** as Sequential Flow Chart (SFC) program. The initialisation routine configures the WTB Link Layer of the i306 interface card and starts the TCN Inauguration.

The link layer configuration data and the WTB inauguration data are prepared in the **POU WTB_Data**. Where the link layer node descriptor values should remain unchanged, the inauguration data may be changed according to the application specific needs. A successful TCN inauguration returns the WTB_Topography object.

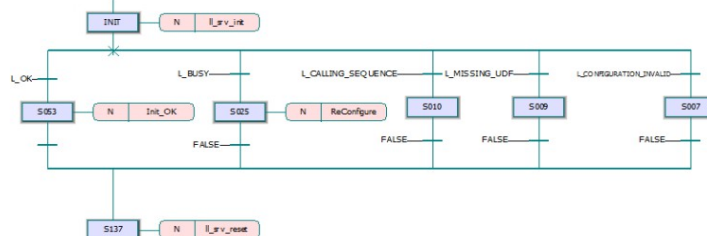
The WTB_Topography Object is read out in the Global_Variables worksheet (see 5.7.4 WTB Configuration).

After a successful TCN inauguration the SFC polls for the WTB Status and WTB Status change requests. The WTB Status Change request commands are mapped to the binary input card i202 channel as shown below:



Thus the WTB status of a node can be changed by means of switches or push buttons connected to module i202. This enables the operator to change the node strength on behalf if required according to the needs of the train composition.

During initialisation as well as when calling WTB status change requests, the return value is evaluated.



In case of a failure, the SFC stops in the appropriate node. A certain application specific action can be performed in this case, e.g. in case of a busy signal, the command can be repeated (see figure below).

WTB Send Data

The process data sent over the WTB is prepared in the **POU WTB_Data**. For this purpose a specific data type for the WTB send data is declared in the MULTIPROG Data_Types declaration work sheet, the struct TYPE_Tx_PD (see below):

```
TYPE
    PD_Array          : ARRAY [0..127] OF USINT;

    TYPE Tx_PD :
    STRUCT
        node : USINT;
        size : USINT;
        data : PD_Array;
    END_STRUCT;
END_TYPE
```

- **node:** Node number of the sending node. (Own node number.)

- **size:** Size of the sent data in bytes.
- **data:** A data buffer of 128 bytes.

Remark:

The information such as node and size are used to address the WTB Driver accordingly. Only the data bytes are sent over WTB all other data is removed by means of the WTB driver. **The structure of Type_Tx_PD must not be changed.**

WTB Receive Data

The process data received over the WTB is prepared in the **POU WTB_Data**. For this purpose a specific data type for the WTB receive data is declared in the MULTIPROG Data_Types declaration work sheet, the struct TYPE_Rx_PD (see below):

```

TYPE
    PD_Array          : ARRAY [0..127] OF USINT;

    Rx_data :
    STRUCT
        node      : USINT;
        size      : USINT;
        freshness : INT;
        data      : PD_Array;
    END_STRUCT;
    Rx_data_array : ARRAY[0..31] OF Rx_data;

    Type_Rx_PD :
    STRUCT
        nr_nodes : USINT;
        data      : Rx_data_array;
    END_STRUCT;
END_TYPE

```

Rx_data:

- **node:** Node number of the sending node. (Other node number:)
- **size:** Size of the received data in bytes.
- **freshness:** Freshness of the received data.
- **data:** A data buffer of 128 bytes.

Type_Rx_data:

- **nr_nodes:** Number of nodes from which data has been received.
- **data:** A data buffer containing receive data for 31 nodes.

Remark:

Only the data bytes are received from the WTB, whereas the other data is added to the data type by means of the WTB driver, in order to give information to the application programmer. **The structure of Type_Rx_PD must not be changed.**

5.7.4 WTB Configuration

The WTB Link Layer services can be called by means of activating one of the following signals in the **Global Variables** worksheet in the section Link Layer Configuration.

Link Layer Configuration					
ll_cfg_transmission_rate	UINT	VAR_GLOBAL	transmission rate in kbit/s, default 1000	%QW20000100	1000
ll_cfg_basic_period	UINT	VAR_GLOBAL	basic period in milliseconds, default 25	%QW20000102	25
ll_cfg_fritting_disabled	UINT	VAR_GLOBAL	=1 if fritting disabled, default 0	%QW20000104	0
ll_cfg_node_descriptor	Type_NodeDescriptor	VAR_GLOBAL	Type_NodeDescriptor	%QD20000106	
ll_cfg_poll_md_when_idle	USINT	VAR_GLOBAL	=1 if background scanning enabled	%QB20000110	0
ll_cfg_sink_port_count	UINT	VAR_GLOBAL	Maximum number of sink ports	%QW20000111	32
ll_cfg_source_port_count	UINT	VAR_GLOBAL	Maximum number of source ports	%QW20000113	1
ll_cfg_port_size	USINT	VAR_GLOBAL	Maximum length of a Port in octets	%QB20000115	128
ll_cfg_max_number_nodes	USINT	VAR_GLOBAL	Maximum numbers of nodes which can be connected	%QB20000124	32
ll_cfg_inaug_data_max_size	USINT	VAR_GLOBAL	Maximum numbers of octets of inaugural data	%QB20000125	124
ll_cfg_inaug_data_size	USINT	VAR_GLOBAL	Actual number of octets of inaugural data	%QB20000126	4
ll_cfg_inaug_data_list	Inaug_data_array	VAR_GLOBAL	Pointer to data area where to connect	%QD20000127	

The used addresses are fixed numbers and must not be changed, otherwise a different service is called.

The returned link layer service state is read out in the Global Variables worksheet, in the section Link Layer Service State

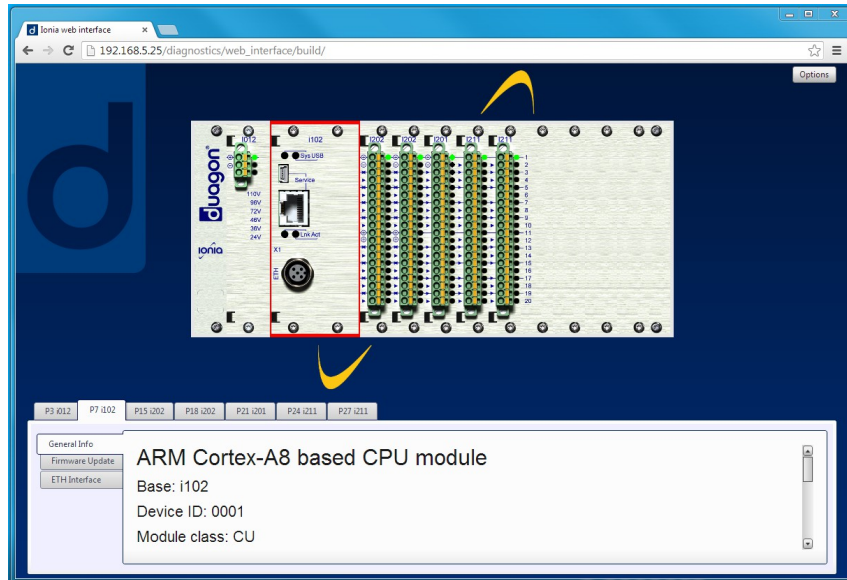
Link Layer Service State				
WTB_STAT_LS_INITIALIZED	BOOL	VAR_GLOBAL	node in state UNCONFIGURED	%MX 3.2000010...
WTB_STAT_LS_CONFIGURED	BOOL	VAR_GLOBAL	node in state CONFIGURED	%MX 3.2000010...
WTB_STAT_LS_READY_TO_NAME	BOOL	VAR_GLOBAL	node in state NAMING_MASTER	%MX 3.2000010...
WTB_STAT_LS_READY_TO_BE_NAMED	BOOL	VAR_GLOBAL	node in state UNNAMED_MAST...	%MX 3.2000010...
WTB_STAT_LS_INHIBIT	BOOL	VAR_GLOBAL	node in Inauguration inhibited	%MX 3.2000010...
WTB_STAT_LS_REGULAR_STRONG	BOOL	VAR_GLOBAL	node in state REGULAR_MAST...	%MX 3.2000010...
WTB_STAT_LS_REGULAR_SLAVE	BOOL	VAR_GLOBAL	node in state REGULAR_SLAVE...	%MX 3.2000010...
WTB_STAT_LS_REGULAR_WEAK	BOOL	VAR_GLOBAL	node in state REGULAR_MAST...	%MX 3.2000010...

With the same method all the TCN Types can be read out and are displayed in the Global Variables work sheet. If complex data types are received than the corresponding structs are expanded in the POU WTB_Data. In the example below, the WTB_TOPO_inauguration_data is expanded as mentioned above.

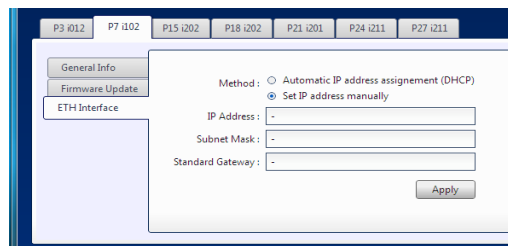
Type_Topography				
WTB_TOPO_node_address	USINT	VAR_GLOBAL		%MB 3.20000200
WTB_TOPO_node_orient	USINT	VAR_GLOBAL		%MB 3.20000201
WTB_TOPO_topo_counter	USINT	VAR_GLOBAL		%MB 3.20000202
WTB_TOPO_individual_period	USINT	VAR_GLOBAL		%MB 3.20000203
WTB_TOPO_is_strong	USINT	VAR_GLOBAL		%MB 3.20000204
WTB_TOPO_number_of_nodes	USINT	VAR_GLOBAL		%MB 3.20000205
WTB_TOPO_bottom_address	USINT	VAR_GLOBAL		%MB 3.20000206
WTB_TOPO_top_address	USINT	VAR_GLOBAL		%MB 3.20000207
WTB_TOPO_inauguration_data	Type_Inauguration_Data	VAR_GLOBAL		%MB 3.20000208

6. IP configuration

The Ethernet interfaces can be configured via the web interface. Start by entering the present IP address of the service Ethernet interface into the web browser and then click on the CPU module:

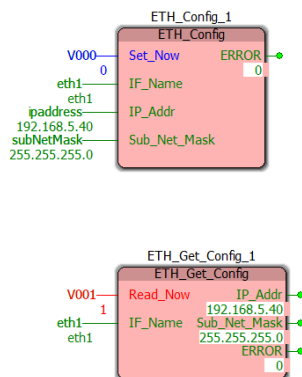


To change the configuration of the service interface (eth0), click on the ETH Interface tab. This opens a dialog where the configuration settings can be entered:



6.0.1 IP configuration from within an IEC61131 program

The ionia library i102_DG_ETH_LIB contains function blocks to get and set the IP address and subnet mask of an Ethernet interface.



For example to configure eth1

Using an ETH_Config FB:

- set IF_Name to 'eth1',
- IP_Addr to the required IP address (e. g. '192.168.5.40')
- and Sub_Net_Mask to the required subnet mask value (e.g. '255.255.255.0').
- As soon as Set_Now is set to TRUE, the settings will be transferred to the ionia, which takes down the appropriate Ethernet interface and brings it back up with the new configuration.

To check the configuration use an ETH_Get_Config FB and:

- set IF_Name to 'eth1'.
- After setting the Read_Now input to TRUE
- the current configuration values appear at the appropriate outputs.

Both FB's use rising edge triggers to synchronise access to the ionia.

7. References

MULTIPROG Express Installer, d-007271-014994

Download link:

http://support.duagon.com/ionia/MULTIPROG_Express_d-007271-015853.exe

Necessary to install the MULTIPROG Express PLC programming tool together with all duagon libraries

ionia i012, i014 short data sheet, d-006282-nnnnnn¹

Short form data sheet of the ionia i012 and i014 power supply modules.

ionia i101, i102, i103 short data sheet, d-006296-nnnnnn¹

Short form data sheet of the ionia i101, i102, i103 cpu modules.

ionia i201/i202 short data sheet, d-006321-nnnnnn¹

Short form data sheet of the ionia i201, i202 digital I/O modules

ionia i205 short data sheet, d-006871-nnnnnn¹

Short form data sheet of the ionia i205 relays module

MVB System User's Guide, d-000656-nnnnnn¹

Introduction and guide to the Multifunction Vehicle Bus (MVB)

IEC61131 API, d-006935-nnnnnn¹

Data sheet of the ionia Multiprog Library elements. These elements are used to access duagon ionia modules.

D2000 MVB Configuration Tool data sheet, d-000341-nnnnnn¹

Data sheet of the D2000 MVB Configuration Tool.

D2000 MVB Configuration Tool, d-000336-nnnnnn¹

D2000 MVB Configuration Tool, is used to build MVB network configurations. The output of a parsed Network configuration file can be downloaded to the ionia TCMS system.

¹ The number "nnnnnn" refers to the latest available revision of this document.

8. Document history

d-007724-014426

New document.

Installer version d-007271-014411

d-007724-015008

Added Ethernet sample application.

Installer version d-007271-014994

d-007724-015105

Added Ethernet configuration.

Installer version d-007271-0nnnnn

Appendix A: Document Numbering System

All Duagon documents have a unique identification number. The identification number has a certain internal structure in order to ease the tracking of different documents. In general, there are two parts:

Prefix	Document number	Filing number
d	-000310	-001952
Always constant	Specifies a certain purpose of a document with the intention to link several documents with different filing number. Please note, that the purpose of the document number is not stored for each document number, but can be derived from the document title, which is stored for each Filing number. The format is either 6 digits or not available.	Unique number, that identifies a particular document. Released in sequential manner as the documents are filed in the archive. A Duagon- internal data base contains exactly one document title text for each filing number. Always 6 digits.

Examples for identification numbers

Identification number	Document Title / Remarks
d-000310-001606	„DXIO data sheet Rev 2.2“
d-000310-001952	„DXIO data sheet Rev 2.3“ A document, that is updated from time to time: the document number has the purpose to link several versions of the „DXIO data sheet“ together. The filing number distinguishes between different versions. Please note, that the document number part is kept the same, as long as the basic intention of the early versions is still kept, for example during revisions due to debugging or manufacturing updates. In case a significant change happens, another document number would be applied.
d-000719	„Notes from prototype meeting ...“ A document, that is obviously not updated after release. The „document number“ part is missing and the filing number remains the only used part for identification.

Recommendation:

In your order, you may specify for example "d-000584-nnnnnn" in order to get the "newest" version of a specific product. When you do not want to follow the sequence of newer versions, i.e. you want to stick to a specific version, then specify the full identification number like "d-000584-002043".